

SMART CONTRACT AUDIT

# Security Review Report

Target: PasswordStore

|              |                                                                                                               |
|--------------|---------------------------------------------------------------------------------------------------------------|
| TARGET       | PasswordStore                                                                                                 |
| REPOSITORY   | <a href="https://github.com/Cyfrin/3-passwordstore-audit">https://github.com/Cyfrin/3-passwordstore-audit</a> |
| PREPARED FOR | Cyfrin / Protocol Team                                                                                        |
| PREPARED BY  | James Lego                                                                                                    |
| DATE         | January 15, 2026                                                                                              |

## Findings/Contents

| Finding Id       | Severity      | Title                                |
|------------------|---------------|--------------------------------------|
| H-1              | High          | Exposure of password stored on-chain |
| H-2              | High          | No Access Controls                   |
| I-1              | Informational | Incorrect Natspec                    |
| Total Findings : | 3             |                                      |

## High

### **HIGH [H-1]** Storing the password on-chain makes it visible to anyone, making it no longer private

**Description:** All data stored on-chain is visible to anyone and can be read directly from the blockchain . The `PasswordStore::s_password` variable is intended to be a private variable and only accessed through the `PasswordStore::getPassword` function , which is intended to be only call by the owner of the contract.

**Impact:** Anyone can read the private password, severely breaking the functionality of the protocol.

**Proof of Concept:** (proof of code)

The below test case shows how anyone can read the password directly from the blockchain.

1. Create a locally running chain

```
$ make anvil
```

1. Deploy the contract to the chain

```
$ make deploy
```

1. Run the storage tool

we use `1` because that's the storage slot of `s_password` in the contract.

```
$ cast storage <contract address> 1 --rpc-url http://127.0.0.1:8545
```

You will get an output that look like this: `0x6d7950617373776f726400000000000000000000000000000000000000000014`

1. You can then parse that hex to a string with:

```
$ cast parse-bytes32-string 0x6d7950617373776f726400000000000000000000000000000000000000000014
```

And get an output like this : `myPassword`

**Recommended Mitigation:** Due to this , the overall architecture of the contract should be rethought. One could encrypt the password off-chain, and then store the encrypted password on-chain. This would require the user to remember another password off-chain to decrypt the password. However, you'd also likely want to remove the view function as you wouldn't want the user to accidentally send a transaction with the password that decrypts the password.

### Likelihood & Impact:

- Impact : High
- Likelihood: High
- Severity : High

**HIGH [H-2]** `PasswordStore::setPassword` has no access controls, meaning a non-owner could change the password.

**Description:** The `PasswordStore::setPassword` function is set to be an `external` function, however, the natspec of the function and overall purpose of the smart contract is that `This function allows only the owner to set a new password.`

```
function setPassword(string memory newPassword) external {
    @> // @audit - There are no access controls
    s_password = newPassword;
    emit SetNetPassword();
}
```

**Impact:** Anyone can set/change the password of the contract, severely breaking the contract intended functionality.

**Proof of Concept:** Add the following to the `PasswordStore.t.sol` test file.

```
function test_non_owner_setting_password(address randomAddress) public {
    vm.assume(randomAddress != owner);
    vm.prank(randomAddress);
    string memory expectedPassword = "myNewPassword";
    passwordStore.setPassword(expectedPassword);

    vm.prank(owner);
    string memory actualPassword = passwordStore.getPassword();
    assertEq(actualPassword, expectedPassword);
}
```

**Recommended Mitigation:** Add an access control conditional to the `setPassword` function.

```
if(msg.sender != s_owner) {
    revert PasswordStore_NotOwner();
}
```

### Likelihood & Impact:

- Impact : High
- Likelihood: High
- Severity : High

## Informational

**INFO [I-1]** The `PasswordStore::getPassword` natspec indicates a parameter that doesn't exist, causing the natspec to be incorrect.

### Description:

```
//@notice This allows only the owner to retrieve the password.  
@> //@param newPassword The new password to set.
```

The `PasswordStore::getPassword` function signature is `getPassword()` while the natspec says it should be `getPassword(string)`.

**Impact:** The natspec is incorrect .

**Recommended Mitigation:** Remove the incorrect natspec line.

```
/*  
 * @notice This allows only the owner to retrieve the password.  
-  * @param newPassword The new password to set.  
  
 */
```

### Likelihood & Impact:

- Impact : None
- Likelihood: None
- Severity : Informational/Gas/Non-Crits

Informational: Hey , this isn't a bug but you should probably fix it.